

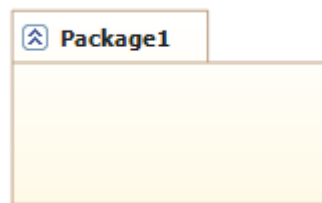


Java中的“包 (package)”

北京理工大学计算机学院
金旭亮

包 (Package)

- Java中的可复用软件资源以包的形式提供
- 每个包都针对某个领域：如网络，GUI（图形用户界面）等。
- 包中存放了彼此在功能上互补的多个类。



包的UML图符

UML，统一建模语言，是面向对象领域用于描述软件系统的一种设计语言，其功能与作用类似于物理电学中的电路图。

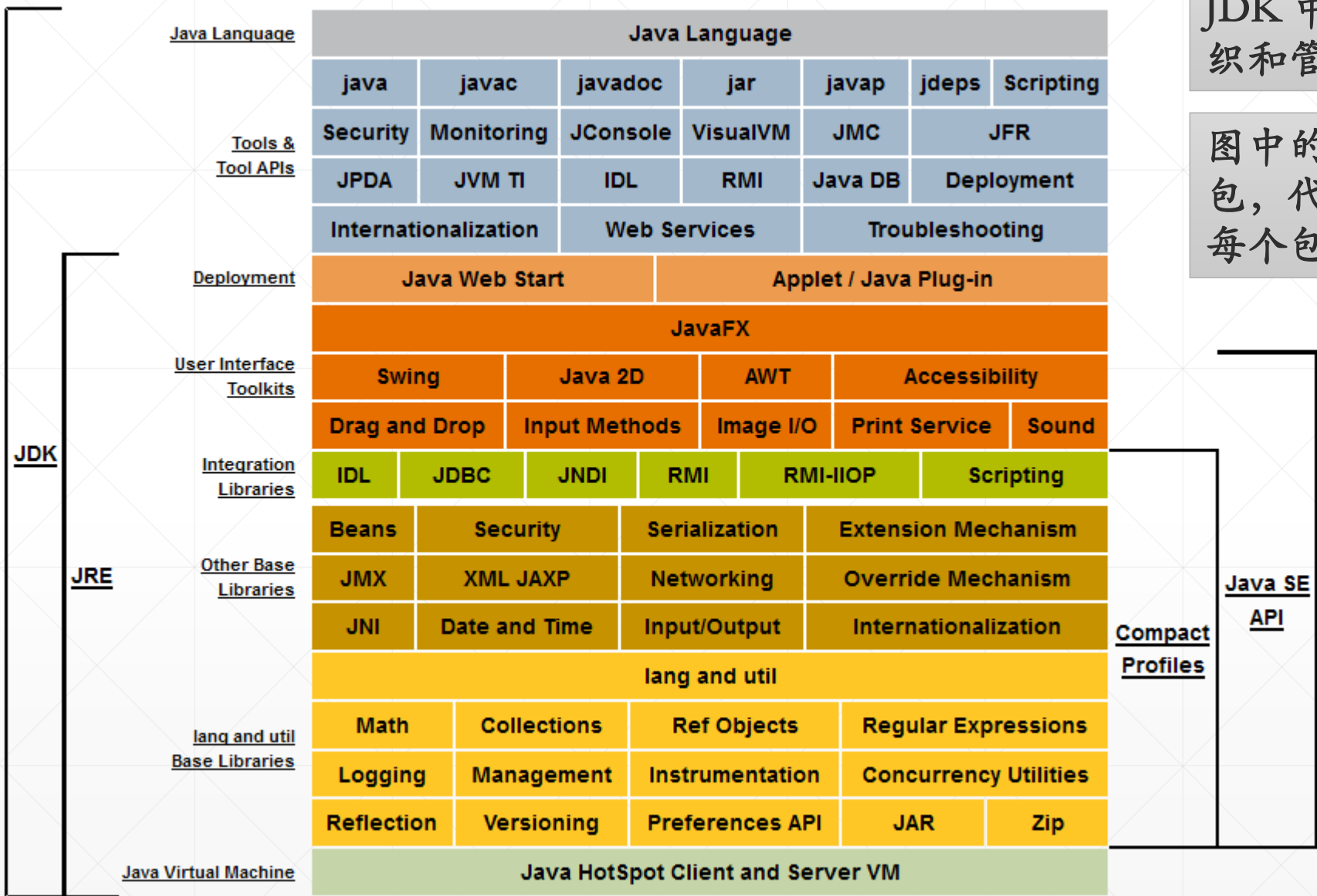


Java “包” 对应于C++/C#中的“命名空间 (name space)”

Java为何要引入“包”这一机制？

1. 解决类的同名问题
2. 便于代码复用与维护





JDK 中的类，都是以包的形式组织和管理起来的。

图中的每个“方块”，都是一个包，代表着一个特定的技术领域，每个包中，包容若干个类和接口。

JDK 8 中的包结构

在实际开发中.....



一个项目，要设计哪些类，构建多少个包，都是需要仔细考虑并尽可能地事先想好的。



在实际开发前，先设计并创建好包，再往里面添加各种类，是常见的作法。



包的创建与使用

掌握Java中包的基本用法

使用包的基本方式—先导入，再使用

- (1) 使用**import**语句仅导入程序中需要使用的特定类型——类或接口

```
import 包名.类型名;
```

- (2) 一次性导入指定包中的所有类型（注意，不支持嵌套导入，也就是说，仅会导入指定包，不会自动导入它的子包。

```
import 包名.*;
```



Java编译器会将import语句引入的包字符串拼接 to 类型标识符前，以得到类型的完整名称。

设定自己所在的包

```
package cn.edu.bit.cs;
```

```
//导入JDK中内置包中的类
```

```
import java.util.Date;
```

```
import javax.swing.JOptionPane;
```

```
public class UseJDKPackage {
```

```
    public static void main(String[] args) {
```

```
        //调用javax.swing包中的JOptionPane的静态方法
```

```
        JOptionPane.showMessageDialog(null, "这是一个消息框");
```

```
        //创建一个java.util包中的Date()对象，获取当前时间
```

```
        String now=new Date().toString();
```

```
        System.out.println("当前时间为: "+now);
```

```
    }
```

```
}
```

使用import语句导入需要
要用到的包中的相应类

推荐使用import语句仅导入需要用到类。
但也可以使用"*"导入特定包中所有的类：
import java.util.*;

示例：UseJDKPackage.java

将类放入自定义包中

在源文件中添加`package`语句可将类加入到指定包中。

```
package cn.edu.bit.cs;  
  
public class HelloWorld {  
  
    public static void main(String[] args) {  
        System.out.println("Hello,World!");  
    }  
}
```

下面通过一些练习来理解包的作用：

假设上述文件（HelloWorld.java）被放到了D:\Test文件夹之下，然后用命令行的方式编译并运行之，会出现哪些问题呢？

编译

用javac的-d参数编译之后，会发现javac自动按照包名创建了相应的文件夹结构，并且把生成的HelloWorld.class文件放到了D:\test\cn\edu\bit\cs下。



注意代码中设定的包名是：
package cn.edu.bit.cs;

```
命令提示符

D:\>cd test

D:\Test>dir *.java
驱动器 D 中的卷是 Working
卷的序列号是 FA4A-1772

D:\Test 的目录

2017/09/09  16:48                146 HelloWorld.java
               1 个文件                146 字节
               0 个目录 159,460,982,784 可用字节

D:\Test>javac -d . *.java

D:\Test>cd cn\edu\bit\cs

D:\Test\cn\edu\bit\cs>dir
驱动器 D 中的卷是 Working
卷的序列号是 FA4A-1772

D:\Test\cn\edu\bit\cs 的目录

2017/09/09  16:51      <DIR>          .
2017/09/09  16:51      <DIR>          ..
2017/09/09  16:51                440 HelloWorld.class
               1 个文件                440 字节
               2 个目录 159,460,982,784 可用字节

D:\Test\cn\edu\bit\cs>
```

运行.....

```
D:\Test\cn\edu\bit\cs>java HelloWorld
错误: 找不到或无法加载主类 HelloWorld

D:\Test\cn\edu\bit\cs>java cn.edu.bit.cs.HelloWorld
错误: 找不到或无法加载主类 cn.edu.bit.cs.HelloWorld

D:\Test\cn\edu\bit\cs>cd \test

D:\Test>java cn.edu.bit.cs.HelloWorld.
Hello, World!
```



请注意在不同文件夹使用java命令执行HelloWorld.class，有时会报告错误：**找不到或无法加载主类**。只有在“根”目录（本例为D:\Test）下，并且指明**完整的类名**，才能正确地运行程序。

Java字节码文件的定位与加载



- ✓ Java从一开始，就规范了代码的命名规范，比如，所有用户自己开发的包，都不能以`java`和`javax`打头，另外，声明为`public`的类，应该放到与其类名一致的文件中，与此同时，以点号分隔的包名，其实对应着相应的文件夹结构，这样一来，给定一个完整的类型名，就可以很快地定位到其编译后的`.class`文件。
- ✓ 所以，当使用`java`命令运行一个`.class`文件时，此文件必须放在与它所在的包名一致的文件夹层次结构之下。
- ✓ 另外，当使用`java`命令运行一个`.class`文件时，JVM会默默地加载它所依赖的JRE中的多个包中的类型，包所在的文件夹可使用“类路径（`class path`）”调整。

```
D:\Test>java -verbose cn.edu.bit.cs.HelloWorld
[Opened D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.lang.Object from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.io.Serializable from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.lang.Comparable from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.lang.CharSequence from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.lang.String from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.lang.reflect.AnnotatedElement from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.lang.reflect.GenericDeclaration from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.lang.reflect.Type from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.lang.Class from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.lang.Cloneable from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.lang.ClassLoader from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.lang.System from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.lang.Throwable from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.lang.Error from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.lang.ThreadDeath from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.lang.Exception from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.lang.RuntimeException from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.lang.SecurityManager from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.security.ProtectionDomain from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.security.AccessControlContext from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.security.SecureClassLoader from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.lang.ReflectiveOperationException from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.lang.ClassNotFoundException from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.lang.LinkageError from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.lang.NoClassDefFoundError from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.lang.ClassCastException from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
[Loaded java.lang.ArrayStoreException from D:\Java\jdk1.8.0_77\jre\lib\rt.jar]
```

使用以下命令可以了解java在运行指定.class文件时到底装载了哪些包中的类型：

java -verbose HelloWorld

可以看到，在JDK8中，绝大多数JRE类型是放在jar包中的，位于JDK\jre\lib文件夹下。

在JDK 15中，相同的命令，输出的信息形式略有变化，给出了加载时间

```
Windows PowerShell
PS D:\test> java -verbose cn.edu.bit.cs.HelloWorld
[0.004s][info][class,load] opened: C:\Program Files\Java\jdk-15\lib\modules
[0.008s][info][class,load] java.lang.Object source: shared objects file
[0.009s][info][class,load] java.io.Serializable source: shared objects file
[0.009s][info][class,load] java.lang.Comparable source: shared objects file
[0.009s][info][class,load] java.lang.CharSequence source: shared objects file
[0.009s][info][class,load] java.lang.constant.Constable source: shared objects file
[0.009s][info][class,load] java.lang.constant.ConstantDesc source: shared objects file
[0.009s][info][class,load] java.lang.String source: shared objects file
[0.010s][info][class,load] java.lang.reflect.AnnotatedElement source: shared objects file
[0.010s][info][class,load] java.lang.reflect.GenericDeclaration source: shared objects file
[0.010s][info][class,load] java.lang.reflect.Type source: shared objects file
[0.010s][info][class,load] java.lang.invoke.TypeDescriptor source: shared objects file
[0.010s][info][class,load] java.lang.invoke.MethodHandle source: shared objects file
[0.010s][info][class,load] java.lang.invoke.MethodHandles source: shared objects file
[0.010s][info][class,load] java.lang.invoke.MethodType source: shared objects file
```

从JDK 9起，Java引入了模块化机制，程序用不上的JRE模块就不会被加载。因此，Java程序的启动性能得到了提升，因为不需要加载过多的类型。

包中类的可访问性



默认情况下，在定义一个类时，如果没有指定这个类为`public`，那它就仅能被同一个包中的其他类型所使用，而不能跨包访问它。



如果要能跨包访问一个类，这个类应该声明为`public`的。



以上描述适用于JDK 8及之前的版本。从JDK 9开始，引入了一个“**模块 (Module)**”系统，情况就发生了变化，如果一个类型所在的包没有被“导出 (`exports`)”，就算它是`public`的，也不能被其他模块中的代码访问。

"类路径"的含义



在前面的例子中，我们看到要想成功地运行一个放在特定包中的Java类，在哪个文件夹下运行java命令是有讲究的。



为了能方便地装载特定Java程序运行时需要的类型，JVM需要知道以哪些文件夹作为“根”，配合要加载的类型所在的包名，去寻找特定类型的.class文件，“**类路径 (class path)**”就是起到了这个作用。



“类路径 (class path)”本质上就是一个“根”文件夹的集合，JVM会到这些文件夹下去查找特定的类型文件。

如何设定类路径?

直接输入java命令，不带任何参数，从输出的信息中可以找到设置类路径的方式：

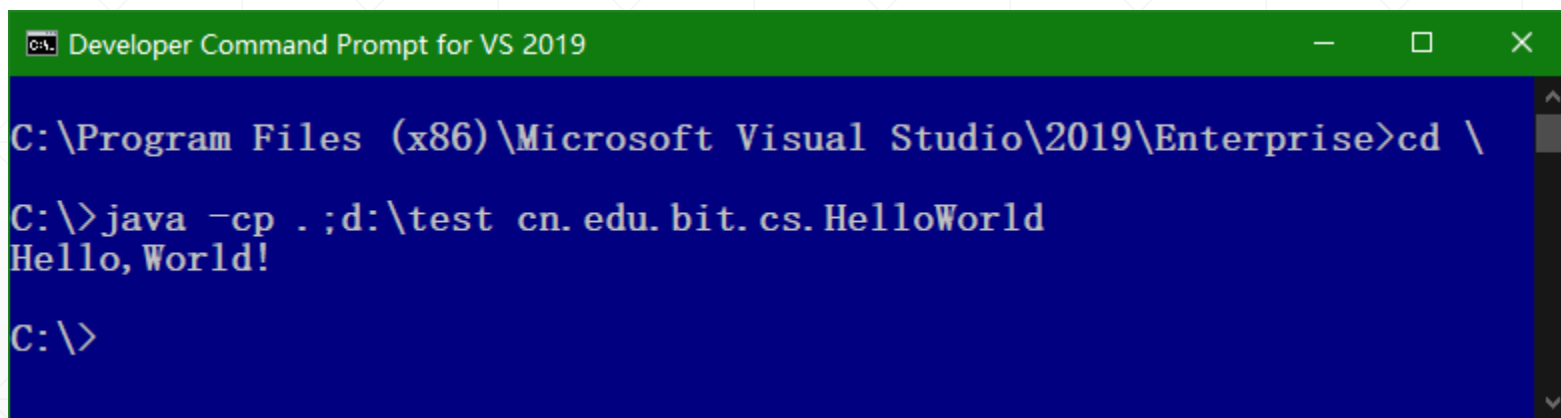
```
-cp <目录和 zip/jar 文件的类搜索路径>  
-classpath <目录和 zip/jar 文件的类搜索路径>  
--class-path <目录和 zip/jar 文件的类搜索路径>  
            使用 ; 分隔的，用于搜索类文件的目录，JAR 档案  
            和 ZIP 档案列表。
```

如果是Windows，在设置路径时，多个文件夹以“分号”分隔，其中“.”表示当前文件夹，“..”表示上一级文件夹。

如果是jar包或zip包，可以直接写出其文件名。

类路径的应用实例

前面例子中，HelloWorld.class文件在d:\test\cn\bit\edu\cs文件夹下，则在其它文件（比如c:\）可以通过-cp参数通知JVM搜索两个地方，一个是“当前文件夹”，用“.”表示，另一个是“d:\test”，后面跟着主类名，就能顺利地运行示例程序：

A screenshot of a Windows Developer Command Prompt window titled "Developer Command Prompt for VS 2019". The window has a green title bar and a dark blue background. The command prompt shows the following sequence of commands and output:

```
C:\Program Files (x86)\Microsoft Visual Studio\2019\Enterprise>cd \  
C:\>java -cp .;d:\test cn.edu.bit.cs.HelloWorld  
Hello, World!  
C:\>
```

下面再来看看如何运行jar中的主类.....

使用放在jar包中的类-1

编写一个测试类：

```
package com.jinxuliang;  
  
public class MyClass {  
    public static void main(String[] args) {  
        System.out.println("MyClass.main()方法运行");  
    }  
}
```

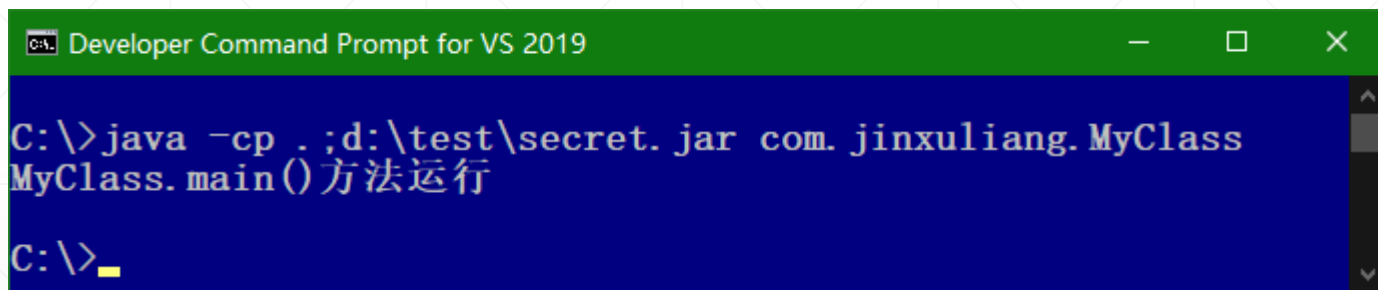
jar包内部包容着一个“完整”的文件夹结构，.class文件就放在里面。

打包生成secret.jar



使用放在jar包中的主类-2

将secret.jar复制到d:\test文件夹下，在另外一个文件夹（比如c:\）下，可以使用以下命令运行它.....



```
C:\>java -cp .;d:\test\secret.jar com.jinxuliang.MyClass  
MyClass.main()方法运行  
C:\>_
```

可以看到，你就把jar包看成是一个文件夹就行了。

运行可执行jar的简便方法



```
Windows PowerShell
PS D:\test> dir *.jar

目录: D:\test

Mode                LastWriteTime         Length Name
----                -
-a----           2020/11/7   20:12         1015 secret.jar

PS D:\test> java -jar secret.jar
MyClass.main()方法运行
PS D:\test> |
```

java另外有一个-jar参数，直接指定jar包文件名即可运行这个jar包，请自己试一试。

下一讲，讨论Java组件化开发的基础技术.....